

Distributed GPU Conjugate Gradient — Communication & Affinity Study

Target: CG-GPU distributed conjugate gradient solver (rocSPARSE / rocBLAS / RCCL + GPU-Aware MPI) **Platform:** single AMD **MI300A** node (1CN192C4G1H_MI300A_Ubuntu22), 4 APU partitions **Toolchain:** ROCm 6.4.1 and 7.2.4, OpenMPI 5.0.10 (UCC/UCX, GPU-Aware) **Matrix:** Dubcova2.pm, 65 536 × 65 536, SPD **Config:** 4 MPI ranks (1 GPU/rank), relative-residual tolerance 1e-6

1. Executive summary

The study built, instrumented, and benchmarked the five communication variants of the CG-GPU solver (staged, isend, rccl, alltoallv_staged, alltoallv). Three findings dominate:

CPU/GPU affinity is the single largest performance factor — far larger than the choice of communication method. Correct rank→GPU→NUMA binding delivered a **~100× reduction in solve time** (e.g. isend 10.6 s → 0.05 s). The default `--gpus=4 --ntasks=4` allocation starved the job of CPU cores and left all ranks sharing a handful of remote hardware threads.

With affinity fixed, communication still dominates the solve at this problem size (~33-84 % of solve time). GPU-Aware transports (isend, alltoallv) and rccl are ~2-3× faster than the host-staged variants, matching the expected cost of the D↔H / H↔D staging copies.

Result correctness and reproducibility were verified. With a fixed RHS seed all five methods converge identically (same initial residual, **172 iterations**, same final residual to 6 digits), confirming the variants differ only in data transport, not numerics.

ROCm 6.4.1 vs 7.2.4: convergence is bit-identical, and **communication is slightly faster on 7.2.4** (notably the dot-product allreduce), but **total solve time is ~1.7-2.2× slower on 7.2.4** at this small per-GPU size, driven by a regression in the *compute* path (rocSPARSE SpMV / rocBLAS / kernel launch). See §5. This is a latency-bound micro-benchmark, so the regression may not carry to larger problems.

rocsparse_v2_spmv does not recover the 7.x regression. Migrating the deprecated `rocsparse_spmv` to the new `rocsparse_v2_spmv` API (persistent SpMV descriptor + one-time symbolic analysis, `csr_adaptive` algorithm) — together with reusing the dense-vector descriptors instead of recreating them per iteration — improved 7.x compute by only **0-3 % (within noise)** and left the ~1.85× gap vs 6.4.1 intact. The regression is therefore in the rocSPARSE **CSR SpMV compute kernel itself**, not in API/descriptor host overhead or the analysis stage. See §5.1.

SDMA copy engines materially reduce communication time. Switching host↔device copies from shader "blit" kernels (`HSA_ENABLE_SDMA=0`) to the dedicated SDMA engines (`=1`, the default) cut halo-exchange time by **8-27 %** on the copy-heavy variants (staged, `alltoallv_staged`, GPU-aware `isend`), with a small further gain from `HSA_ENABLE_SDMA_GANG=1`. Compute was unchanged, confirming the effect is isolated to data transport. See §5.2.

2. Methodology

Artifacts produced during the study (all in CG-GPU/):

File	Purpose
Makefile	Build <code>cg_gpu</code> (auto-detects <code>--offload-arch</code> , links rocSPARSE/rocBLAS/RCCL/HIP). <code>make SPMV_V2=1</code> selects the <code>rocsparse_v2_spmv</code> path.
test_cg_gpu.sh	Build + run all 5 variants, verify convergence, print timing summary.

File	Purpose
submit_cg_gpu_test.sbatch	SLURM wrapper (--exclusive + binding) to build/run on a GPU node.
sweep_rocm_versions.sh submit_rocm_sweep.sbatch	/ Sweep ROCm versions (one method) in a single allocation; supports SPMV="v1 v2" to build+benchmark both SpMV paths.
sweep_sdma.sh / submit_sdma_sweep.sbatch	Sweep SDMA vs blit copy engines (HSA_ENABLE_SDMA / _GANG) across all methods, one build.
gpu_bind.sh	Per-rank GPU + NUMA binding wrapper.
check_affinity.cpp submit_affinity_check.sbatch	/ Diagnostic: per-rank GPU PCI id + CPU mask.

Instrumentation added to src/cg.cpp. Two wall-clock accumulators, reduced as the max across ranks and reset at the start of the timed CG loop (setup/I/O excluded):

- `g_halo_time` — SpMV ghost exchange: staging copies, GPU gather/pack, and MPI/RCCCL send/recv/wait/alltoallv (excludes the rocSPARSE SpMV compute).
- `g_allreduce_time` — the dot-product MPI_Allreduce.

The solver now reports `comm total`, `halo exchange`, `dot allreduce`, and `compute (rest)` alongside `CG solve time`. A `CG_SEED` runtime option (env var or `argv[3]`) fixes the RHS for reproducible runs.

SpMV path (src/cg.cpp). The dense-vector (`rocsparse_dnvec`) descriptors are created once per matrix block and rebound each iteration with `rocsparse_dnvec_set_values` (previously they were created/destroyed on every SpMV). A compile-time `USE_V2_SPMV` flag (make `SPMV_V2=1`) switches the compute call from the deprecated `rocsparse_spmv` to `rocsparse_v2_spmv`: a persistent `rocsparse_spmv_descr` with `csr_adaptive`, a one-time blocking `rocsparse_v2_spmv_stage_analysis` at upload, and `rocsparse_v2_spmv_stage_compute` in the loop. An `__has_include` guard falls back to the classic path on ROCm 6.x (v2 does not exist there).

Convergence test. Relative residual in the 2-norm: iterate until $\|r_k\|_2 \leq 1e-6 * \|r_0\|_2$, capped at `max_iter = 1.3*N + 2`, with the true residual recomputed every 8 iterations.

3. Affinity study (the dominant effect)

Topology (MI300A)

`rocm-smi + lscpu` confirm **GPU $i \leftrightarrow$ NUMA node i** , all GPUs linked by XGMI (1 hop):

NUMA	CPUs	GPU (PCI)
0	0-23, 96-119	GPU0 0000:01
1	24-47, 120-143	GPU1 0001:01
2	48-71, 144-167	GPU2 0002:01
3	72-95, 168-191	GPU3 0003:01

GPU assignment

The solver's `hipSetDevice(rank % num_gpus)` already gives **distinct physical GPUs** under `mpirun` (verified via distinct PCI bus ids). Setting `ROCR_VISIBLE_DEVICES` per rank is **not required for correctness** with `mpirun`. Two caveats surfaced:

- **srun without PMIx** launches each task as its own MPI world of size 1 → every task picks GPU 0 → **all ranks collide on one GPU**. Use `srun --mpi=pmix` or the `gpu_bind.sh`

wrapper.

- **Multi-node** runs use the global rank in the modulo; a node-local rank is more robust.

CPU/NUMA binding — measured impact

Launch	GPU/rank	CPU affinity	staged	isend	rccl	a2a_staged	alltoallv
--bind-to none, not distinct exclusive		4 hwthreads shared by all ranks (remote) each rank	9.27 s	10.58 s	7.39 s	9.00 s	8.46 s
--exclusive + gpu_bind.sh	distinct	→ GPU-local NUMA (24 cores)	0.15 s	0.05 s	0.05 s	0.09 s	0.05 s

Root cause: without `--exclusive/--cpus-per-task`, SLURM handed the whole job only ~4 hardware threads (in NUMA 2/3), and `--bind-to none` let all four ranks contend for them — remote from GPU0/GPU1. Every `hipDeviceSynchronize`, kernel launch, and MPI progress call serialized on contended, remote cores. GPU assignment was never the bottleneck; **host-side CPU locality was**.

Recommended launch: `mpirun -n 4 ./gpu_bind.sh ./cg_gpu <matrix> <method>` inside an `--exclusive` allocation. The wrapper pins each rank to one GPU (`ROCR_VISIBLE_DEVICES=<local_rank>`) and its NUMA-local CPUs/memory (`numactl --cpunodebind --membind`).

4. Communication timing results (fixed seed = 12345)

All methods solve the identical system (init residual 196.121, **172 iterations**, final residual 1.815e-4).

Convergence verification (job 13149, CG_SEED=12345)

With the seed fixed, every method produces bit-identical convergence — same initial residual, iteration count, and final residual — confirming the variants differ only in data transport, not numerics:

method	seed	init resid	iters	final resid	solve (s)	comm (s)
staged	12345	196.121	172	0.000181523	0.148	0.124
isend	12345	196.121	172	0.000181523	0.051	0.028
rccl	12345	196.121	172	0.000181523	0.051	0.025
alltoallv_staged	12345	196.121	172	0.000181523	0.082	0.056
alltoallv	12345	196.121	172	0.000181523	0.051	0.025

Timing breakdown

Times are seconds, max across ranks, CG loop only:

method	solve	comm total	halo exch	dot allreduce	compute	comm %
staged	0.1477	0.1243	0.1095	0.0148	0.0234	84 %
isend	0.0513	0.0282	0.0142	0.0140	0.0231	55 %

method	solve	comm total	halo excl	dot allreduce	compute	comm %
rccl	0.0505	0.0252	0.0160	0.0092	0.0253	50 %
alltoallv_staged	0.0823	0.0560	0.0422	0.0138	0.0263	68 %
alltoallv	0.0512	0.0250	0.0129	0.0121	0.0262	49 %

Interpretation

- **Staging is expensive.** staged halo (0.110 s) vs isend (0.014 s) — a $\sim 7\times$ gap that is purely the D $\rightarrow$ H + H $\rightarrow$ D copies GPU-Aware MPI removes. Similarly alltoallv_staged (0.042 s) vs alltoallv (0.013 s) isolates the PCIe round-trip cost.
- **GPU-Aware point-to-point, collective, and RCCL are comparable** here (~ 0.013 – 0.016 s halo). At 4 GPUs on one node over XGMI, none has a decisive advantage; differences are within run-to-run noise.
- **Communication dominates** (49–84 % of solve). The per-GPU sub-problem (~ 16 k rows) is small, so kernel launch latency, MPI_Allreduce latency, and ghost exchange outweigh floating-point work.
- **dot allreduce is a real cost** (0.009–0.015 s, up to 27 % of solve): two latency-bound scalar allreduces per iteration, each preceded by a hipDeviceSynchronize.

Caveats

- For overlap variants (isend, rccl, alltoallv) the reported halo time is mostly the **exposed wait**; on-proc SpMV runs concurrently, so this is not the raw wire time.
- Single node / single matrix / single rank count. Absolute times are small and latency-bound; treat cross-method deltas as indicative, not definitive.

5. ROCm version comparison (6.4.1 vs 7.2.4)

Identical study (same node class, 4 ranks, --exclusive + gpu_bind.sh, CG_SEED=12345), rebuilt from source under each toolchain. Two clean runs per version; values below are the run-average in seconds (compute = solve – comm). Both versions converge **bit-identically** (172 iterations, final residual $1.815e-4$), so only performance differs.

method	solve 6.4.1	solve 7.2.4	Δ solve	compute e 6.4.1	compute e 7.2.4	comm 6.4.1	comm 7.2.4	allredu ce 6.4.1	allredu ce 7.2.4
staged	0.152	0.261	+72 %	0.023	0.113	0.129	0.148	0.016	0.014
isend	0.051	0.111	+118 %	0.024	0.091	0.028	0.020	0.014	0.003
rccl	0.051	0.110	+116 %	0.027	0.093	0.024	0.017	0.010	0.004
alltoallv_staged	0.082	0.138	+68 %	0.021	0.085	0.061	0.053	0.018	0.011
alltoallv	0.059	0.114	+93 %	0.031	0.097	0.028	0.017	0.009	0.003

Findings

- **Numerics unchanged.** Same iteration count and residual to 6 digits on both toolchains.
- **Communication is as good or better on 7.2.4.** The dot-product MPI_Allreduce is ~ 3 – $4\times$ cheaper (e.g. isend 0.014 s $\rightarrow$ 0.003 s), and GPU-Aware halo times are comparable or slightly lower. The MPI / RCCL path did not regress.
- **The compute path regressed $\sim 4\times$** (e.g. isend compute 0.024 s $\rightarrow$ 0.091 s), and since compute now dominates, **total solve time roughly doubled**. "Compute" here is rocSPARSE SpMV + rocBLAS (ddot/daxpy/dscal) + kernel-launch/host overhead.
- **rocsparse_spmv is deprecated in 7.2.4** (compiler warns to use rocsparse_v2_spmv). The deprecated path and/or increased per-call host overhead in the newer rocSPARSE/rocBLAS is the most likely cause; this benchmark is latency-bound at ~ 16 k rows/GPU, so host-side overheads dominate.

- **Binaries are not runtime-compatible across major versions.** A 7.2.4-linked `cg_gpu` run against the 6.4.1 HSA runtime failed with undefined symbol: `hsa_and_memory_get_preferred_copy_engine` — rebuild per toolchain (as the test does).

Caveats

Small, latency-bound problem; single node; two runs per version. The compute regression should be re-checked at larger per-GPU problem sizes. The `rocsparse_v2_spmv` follow-up is reported in §5.1. A version-labeled column could be added to `test_cg_gpu.sh` output for automated regression tracking.

5.1 rocsparse_v2_spmv follow-up (does it recover the 7.x regression?)

§5 attributed the 7.x slowdown to the compute path and flagged the deprecated `rocsparse_spmv`. To test whether the API itself was responsible, the SpMV was reworked two ways (both in `src/cg.cpp`):

1. **Persistent dense-vector descriptors** (applied to *both* paths) — create `rocsparse_dnvec` descriptors once and rebind pointers with `rocsparse_dnvec_set_values`, removing two descriptor create/destroy calls per iteration.
2. **rocsparse_v2_spmv** (make `SPMV_V2=1`) — persistent `rocsparse_spmv_descr`, `csr_adaptive` algorithm, one-time `..._stage_analysis` at upload, `..._stage_compute` in the loop.

Sweep across the 7.x line (method `isend`, 4 ranks, `--exclusive + gpu_bind.sh`, `CG_SEED=12345`, **minimum of 5 runs**; 6.4.1 v1 as baseline; `sweep_rocm_versions.sh` `SPMV="v1 v2"`, job 13155). All rows converge in 172 iterations, so only performance differs:

ROCM	SpMV path	iters	solve (s)	comm (s)	compute (s)
6.4.1	v1 (rocsparse_sp mv)	172	0.0480	0.0178	0.0302 (baseline)
7.0.2	v1 (rocsparse_sp mv)	172	0.0717	0.0148	0.0569
7.0.2	v2 (rocsparse_v2_ spmv)	172	0.0707	0.0160	0.0547
7.2.4	v1 (rocsparse_sp mv)	172	0.0700	0.0141	0.0559
7.2.4	v2 (rocsparse_v2_ spmv)	172	0.0708	0.0150	0.0558
7.13.0	v1 (rocsparse_sp mv)	172	0.0713	0.0146	0.0567
7.13.0	v2 (rocsparse_v2_ spmv)	172	0.0720	0.0168	0.0552

Findings

- **v2 does not fix the regression.** v2 improves 7.x compute by only **0–3 %** (7.0.2 0.0569 → 0.0547; 7.2.4 essentially unchanged; 7.13.0 0.0567 → 0.0552) — all within run-to-run noise. Compute on every 7.x release remains **~1.85× the 6.4.1 baseline** (~0.055 s vs 0.030 s).
- **The persistent-dnvec fix is now in the v1 numbers too**, and v1 on 7.x is still slow — so per-call descriptor/host overhead was *not* the cause. The one-time analysis stage of v2 also rules out repeated in-kernel preprocessing as the culprit.
- **Conclusion: the regression lives in the rocSPARSE CSR SpMV compute kernel**

(and/or the surrounding rocBLAS/launch path), not in the SpMV API surface. Migrating to rocsparse_v2_spmv is still worthwhile for API longevity (v1 is deprecated) but is not a performance remedy here.

- The regression is **flat across the entire 7.x line** (7.0.2 → 7.13.0), i.e. introduced at the 6.x→7.x boundary and not since improved.

Next step to localize it

A one-iteration rocprofv3 kernel trace on 6.4.1 vs 7.2.4 (same binary flags, same seed) to identify the specific SpMV kernel and dispatch whose duration grew, and to check the csr_adaptive vs csr_rowsplit algorithm choice for this near-uniform-nnz matrix.

5.2 SDMA vs blit-kernel copy engines

The HSA runtime can service host↔device (and some device↔device) copies with either the dedicated **SDMA / DMA engines** or with shader "**blit**" **kernels**. Two runtime flags control this:

- HSA_ENABLE_SDMA=0 — force blit kernels (shader copies); =1 — use SDMA engines (**runtime default**).
- HSA_ENABLE_SDMA_GANG=1 — gang multiple SDMA engines to service one copy.

This touches only the *copy* path, so it should move communication time (halo exchange, especially the host-staged variants' D↔H copies) while leaving the rocSPARSE/rocBLAS compute time unchanged. The binary is identical across configs — only the runtime environment varies (sweep_sdma.sh / submit_sdma_sweep.sbatch).

Sweep on **ROCm 6.4.3**, 4 ranks, --exclusive + gpu_bind.sh, CG_SEED=12345, **minimum of 5 runs** (job 13156). blit = SDMA off, sdma = SDMA on (default), sdma_gang = SDMA on + gang. All rows converge in 172 iterations; times are seconds, max across ranks, CG loop only:

method	config	solve	comm	halo exch	dot allreduce	compute	halo Δ vs blit
staged	blit	0.1575	0.1463	0.1215	0.0248	0.0112	—
staged	sdma	0.1418	0.1245	0.1116	0.0129	0.0173	−8 %
staged	sdma_gang	0.1482	0.1264	0.1129	0.0136	0.0218	−7 %
isend	blit	0.0483	0.0260	0.0154	0.0106	0.0223	—
isend	sdma	0.0496	0.0261	0.0119	0.0142	0.0235	−23 %
isend	sdma_gang	0.0454	0.0240	0.0113	0.0127	0.0214	−27 %
rccl	blit	0.0480	0.0240	0.0143	0.0097	0.0240	—
rccl	sdma	0.0464	0.0244	0.0129	0.0115	0.0220	−10 %
rccl	sdma_gang	0.0478	0.0246	0.0129	0.0117	0.0232	−10 %
alltoallv_staged	blit	0.0863	0.0685	0.0480	0.0204	0.0178	—
alltoallv_staged	sdma	0.0777	0.0599	0.0418	0.0181	0.0178	−13 %
alltoallv_staged	sdma_gang	0.0788	0.0630	0.0394	0.0236	0.0158	−18 %
alltoallv	blit	0.0399	0.0167	0.0116	0.0051	0.0232	—
alltoallv	sdma	0.0491	0.0249	0.0140	0.0109	0.0242	+21 %
alltoallv	sdma_gang	0.0497	0.0240	0.0121	0.0120	0.0257	+4 %

Findings

- **SDMA helps communication, as predicted.** Disabling it (blit kernels) is slower on every copy-heavy path: host-staged staged (−8 % halo) and alltoallv_staged (−13...−18 %),

and GPU-aware isend (-23...-27 %). Blit copies consume CU/shader cycles and contend with the SpMV kernels; SDMA offloads them to dedicated engines.

- **Ganging adds a small extra win** on the copy-bound variants (isend -23 %→-27 %, alltoallv_staged -13 %→-18 %) but is within noise for rccl/staged.
- **Compute is independent of the flag**, exactly as expected — the compute column (≈ 0.011 - 0.026 s) shows no systematic trend with the SDMA setting; the spread is run noise absorbed by the solve - comm residual. The copy-engine choice moves only the communication side, confirming the effect is isolated to data transport.
- **One exception:** the tiny GPU-aware alltoallv was marginally *best with blit* (halo 0.0116 vs 0.0140). At these very small message sizes SDMA engine setup latency can exceed a quick shader copy; the effect is small and near noise.

Net: the ROCm default (HSA_ENABLE_SDMA=1) is the right choice here — forcing blit (=0) costs up to ~25 % of halo time on the staged and GPU-aware point-to-point paths.

HSA_ENABLE_SDMA_GANG=1 is a minor additional win for the copy-heaviest variants and is worth enabling when the halo volume is large.

6. Recommendations — further optimization

A. Reduce dot-product synchronization (highest ROI). Each iteration does two separate MPI_Allreduces, each gated by a full hipDeviceSynchronize. Options: - Fuse the two dot products (r*r and p*Ap) into one allreduce where the algorithm allows. - Adopt a **pipelined / communication-avoiding CG** (e.g. Ghysels & Vanroose pipelined CG, or s-step CG) to overlap the global reduction with SpMV and cut the number of synchronization points. - Replace hipDeviceSynchronize with stream/event-scoped sync so unrelated GPU work is not stalled.

B. Cut kernel-launch overhead. At ~16k rows/GPU the loop is launch-latency bound. Consider **HIP graphs** to capture the per-iteration kernel/BLAS sequence, or fuse the rocBLAS axpy/scal/dot chain into custom kernels to reduce launches.

C. Improve SpMV/comm overlap. - In rccl, replace the full hipDeviceSynchronize after the gather with an **event + hipStreamWaitEvent** so only rccl_stream waits (comment in the code already flags this). - For the point-to-point variants, split on-proc SpMV to better hide MPI_Waitall latency.

D. Determinism (optional). If bit-reproducible residuals across methods are required, pin rocSPARSE SpMV to a non-atomic algorithm and use deterministic reductions; document the (small) performance trade-off.

E. Packaging. - Detect --offload-arch per target more robustly; the current fallback is gfx942. - Bake the affinity guidance into the default launch path so users don't hit the un-bound slow path.

F. Track the ROCm 7.2.4 compute regression. - ~Migrate rocsparse_spmv → rocsparse_v2_spmv and re-measure. ~ **Done (§5.1):** v2 does *not* recover the regression (0-3 %, within noise). The cause is the rocSPARSE CSR SpMV kernel, not the API — so keep the v2 migration for API longevity but pursue the kernel-level cause separately. - **Localize with rocprofv3:** one-iteration kernel trace on 6.4.1 vs 7.2.4 to find the SpMV kernel whose duration grew; file a rocSPARSE issue with the reproducer if confirmed. - Try rocsparse_spmv_alg_csr_rowsplit for this near-uniform-nnz matrix (currently csr_adaptive). - Add a version-labeled timing column to test_cg_gpu.sh for automated cross-version regression tracking. - Re-run the comparison at larger per-GPU sizes to see whether the regression is confined to the latency-bound regime.

7. Recommendations — further analysis

1. **Strong & weak scaling.** Sweep 1→4 GPUs (single node) and, if available, 2+ nodes to expose inter-node vs intra-node (XGMI) behavior — where rccl and GPU-Aware alltoallv should diverge.
2. **Problem-size sweep.** Larger matrices per GPU will shift the compute/comm balance; report a compute-bound regime to complement the current latency-bound one.
3. **Isolate exposed vs. raw communication.** Add non-overlapped timing (or ROC-profiler / rocprofv3 traces) to separate wire time from exposed wait for the overlap variants.
4. **Statistical rigor.** Repeat each method N times with a fixed seed and report median ± spread; current deltas are within noise for the fast variants.
5. **Profiler-level attribution.** Use rocprof/rocprofiler-systems to attribute time to SpMV,

dot, allreduce, and copies, and to confirm the NUMA-locality benefit at the hardware-counter level.

6. **Communication volume model.** Correlate measured halo time with the actual ghost-exchange byte counts per rank (from `form_comm`) to build a latency+bandwidth model.
-

8. Reproduce

```
cd CG-GPU
```

```
# Full benchmark on a compute node (exclusive + affinity), fixed seed:
sbatch --export=ALL,CG_SEED=12345 submit_cg_gpu_test.sbatch
```

```
# Same study on a specific ROCm version (run jobs one at a time -- concurrent
# jobs in the same directory race on the shared cg_gpu binary):
sbatch --export=ALL,CG_SEED=12345,ROCM_MODULE=rocm/7.2.4
submit_cg_gpu_test.sbatch
```

```
# Affinity / GPU-assignment diagnostic:
sbatch submit_affinity_check.sbatch
```

```
# ROCm-version sweep comparing the two SpMV paths (v1 vs v2), one method:
sbatch --export=ALL,SPMV="v1 v2",VERSIONS="6.4.1 7.0.2 7.2.4
7.13.0",REPEATS=5,METHOD=isend \
submit_rocm_sweep.sbatch
```

```
# SDMA vs blit copy-engine sweep (all methods, one build):
sbatch --export=ALL,ROCM_VER=6.4.3,REPEATS=5 submit_sdma_sweep.sbatch
```

```
# Single method, reproducible (v1 path):
CG_SEED=12345 mpirun -n 4 ./gpu_bind.sh ./cg_gpu src/Dubcova2.pm rccl
```

```
# Build the rocsparse_v2_spmv path explicitly:
make clean && make SPMV_V2=1
```

Environment: `module load rocm/6.4.1` and a GPU-Aware openmpi build (see `README.md`).